

Module 07 — Agrégations et regroupements

Bootcamp Data Analyst — From Zero to Hero | Niveau Débutant · Acte III

Ce que tu seras capable de faire à la fin de ce module

- Utiliser les 5 fonctions d'agrégation : `COUNT`, `SUM`, `AVG`, `MIN`, `MAX`
- Arrondir les résultats avec `ROUND()`
- Regrouper des données avec `GROUP BY` pour comparer des catégories
- Filtrer des groupes avec `HAVING`
- Comprendre la différence entre `WHERE` et `HAVING`
- Maîtriser l'ordre d'exécution SQL pour éviter les erreurs classiques
- Écrire des requêtes complètes de type reporting business

 **Durée estimée :** 40 minutes

 **Prérequis :** Module 06 complété — tu sais utiliser `SELECT`, `WHERE`, `ORDER BY`, `LIMIT`

 **Environnement :** sqliteonline.com — zéro installation

1. Pourquoi les agrégations ?

Tu travailles comme Data Analyst junior chez un opérateur télécom à Abidjan. Ton DG te convoque :

« J'ai besoin des chiffres pour le comité de direction de vendredi. Quel agent a généré le plus de chiffre d'affaires ce mois-ci ? Quelle région est la moins performante ? Et combien de transactions ont dépassé 2 000 FCFA ? »

Tu as une table de **10 000 transactions** dans ta base de données.

Avec ce que tu as appris au Module 06, tu peux lire les lignes une par une. Mais lire 10 000 lignes pour répondre à ces 3 questions ? **Impossible.**

Ce que tu faisais au M06 — ligne à ligne

SQL

```
SELECT agent, montant
FROM transactions
WHERE region = 'Abidjan';
```

→ Résultat : des centaines de lignes à parcourir manuellement.

Ce que tu vas faire maintenant — résumé en une ligne

SQL

```
SELECT agent, SUM(montant) AS chiffre_affaires
FROM transactions
GROUP BY agent
ORDER BY chiffre_affaires DESC
LIMIT 1;
```

→ Résultat : **1 ligne** — l'agent champion avec son total.

💡 Les agrégations, c'est le passage du "je lis les données" au "je produis des insights". C'est là que le Data Analyst commence vraiment à créer de la valeur.

2. Le dataset fil rouge

Pour tout ce module, on travaille avec la table `transactions` d'un opérateur télécom fictif — inspiré du contexte ANSUT-CI.

Structure de la table :

COLONNE	TYPE	DESCRIPTION
<code>id_transaction</code>	INTEGER	Identifiant unique de la transaction
<code>id_client</code>	INTEGER	Identifiant du client
<code>agent</code>	TEXT	Nom de l'agent commercial
<code>region</code>	TEXT	Région (Abidjan, Bouaké, San Pedro, Yamoussoukro)
<code>type_produit</code>	TEXT	Produit vendu (Recharge_500, Recharge_1000, Forfait_internet, Forfait_voix)
<code>montant</code>	INTEGER	Montant en FCFA
<code>mois</code>	TEXT	Mois de la transaction (2024-01, 2024-02...)

Extrait des 15 premières lignes :

ID_TRANSACTION	ID_CLIENT	AGENT	REGION	TYPE_PRODUIT	MONTANT	MOIS
1	1042	Koné Mamadou	Abidjan	Forfait_internet	2 000	2024-01
2	2317	Traoré Aminata	Bouaké	Recharge_500	500	2024-01
3	1890	Kouassi Éric	Abidjan	Recharge_1000	1 000	2024-01
4	3451	N'Guessan Fatou	San Pedro	Forfait_voix	1 500	2024-01
5	2100	Koné Mamadou	Abidjan	Recharge_500	500	2024-01
6	1230	Bamba Seydou	Yamoussoukro	Forfait_internet	2 000	2024-02
7	4012	Traoré Aminata	Bouaké	Forfait_internet	2 000	2024-02
8	1042	Kouassi Éric	Abidjan	Recharge_1000	1 000	2024-02
9	5500	N'Guessan Fatou	San Pedro	Recharge_500	500	2024-02
10	2800	Koné Mamadou	Abidjan	Forfait_voix	1 500	2024-02
11	3100	Bamba Seydou	Yamoussoukro	Recharge_1000	1 000	2024-02
12	1750	Traoré Aminata	Bouaké	Recharge_500	500	2024-03
13	2200	Kouassi Éric	Abidjan	Forfait_internet	2 000	2024-03
14	4800	N'Guessan Fatou	San Pedro	Forfait_voix	1 500	2024-03
15	1100	Koné Mamadou	Abidjan	Recharge_500	500	2024-03

 La table complète contient ~500 lignes — elle sera fournie dans le Mini-projet B. Pour ce module, on travaille sur un extrait de 20 lignes.

Créer la table de test dans sqliteonline.com

Colle ce script dans sqliteonline.com et exécute-le :

SQL

```
CREATE TABLE transactions (  
  id_transaction INTEGER PRIMARY KEY,  
  id_client INTEGER,  
  agent TEXT,  
  region TEXT,  
  type_produit TEXT,  
  montant INTEGER,  
  mois TEXT  
);  
  
INSERT INTO transactions VALUES  
  (1, 1042, 'Koné Mamadou', 'Abidjan', 'Forfait_internet', 2000, '2024-01'),  
  (2, 2317, 'Traoré Aminata', 'Bouaké', 'Recharge_500', 500, '2024-01'),  
  (3, 1890, 'Kouassi Éric', 'Abidjan', 'Recharge_1000', 1000, '2024-01'),  
  (4, 3451, 'N'Guessan Fatou', 'San Pedro', 'Forfait_voix', 1500, '2024-01'),  
  (5, 2100, 'Koné Mamadou', 'Abidjan', 'Recharge_500', 500, '2024-01'),  
  (6, 1230, 'Bamba Seydou', 'Yamoussoukro', 'Forfait_internet', 2000, '2024-02'),  
  (7, 4012, 'Traoré Aminata', 'Bouaké', 'Forfait_internet', 2000, '2024-02'),  
  (8, 1042, 'Kouassi Éric', 'Abidjan', 'Recharge_1000', 1000, '2024-02'),  
  (9, 5500, 'N'Guessan Fatou', 'San Pedro', 'Recharge_500', 500, '2024-02'),  
  (10, 2800, 'Koné Mamadou', 'Abidjan', 'Forfait_voix', 1500, '2024-02'),  
  (11, 3100, 'Bamba Seydou', 'Yamoussoukro', 'Recharge_1000', 1000, '2024-02'),  
  (12, 1750, 'Traoré Aminata', 'Bouaké', 'Recharge_500', 500, '2024-03'),  
  (13, 2200, 'Kouassi Éric', 'Abidjan', 'Forfait_internet', 2000, '2024-03'),  
  (14, 4800, 'N'Guessan Fatou', 'San Pedro', 'Forfait_voix', 1500, '2024-03'),  
  (15, 1100, 'Koné Mamadou', 'Abidjan', 'Recharge_500', 500, '2024-03'),  
  (16, 2900, 'Bamba Seydou', 'Yamoussoukro', 'Forfait_voix', 1500, '2024-03'),  
  (17, 3800, 'Traoré Aminata', 'Bouaké', 'Forfait_internet', 2000, '2024-03'),  
  (18, 1600, 'Kouassi Éric', 'Abidjan', 'Recharge_500', 500, '2024-03'),
```

```
(19, 4200, 'N' 'Guessan Fatou', 'San Pedro', 'Recharge_1000', 1000, '2024-03'),
(20, 5100, 'Koné Mamadou', 'Abidjan', 'Forfait_internet', 2000, '2024-03');
```

3. Les 5 fonctions d'agrégation

Une **fonction d'agrégation** prend un ensemble de valeurs (une colonne) et retourne **une seule valeur résumée**.

3.1 COUNT() — Compter

Rôle : compte le nombre de lignes ou de valeurs non nulles.

VARIANTE 1 : **COUNT(*)** — COMPTER TOUTES LES LIGNES

```
SQL
SELECT COUNT(*) AS total_transactions
FROM transactions;
```

TOTAL_TRANSACTIONS
20

COUNT(*) compte **toutes les lignes**, y compris celles avec des valeurs NULL.

VARIANTE 2 : **COUNT(COLONNE)** — COMPTER LES VALEURS NON NULLES

```
SQL
SELECT COUNT(agent) AS nb_agents_reseignes
FROM transactions;
```

`COUNT(colonne)` ignore les NULL dans cette colonne. Si 3 lignes n'ont pas d'agent renseigné, `COUNT(agent)` retourne 17 — pas 20.

VARIANTE 3 : `COUNT(DISTINCT COLONNE)` — COMPTER LES VALEURS UNIQUES

SQL

```
SELECT COUNT(DISTINCT agent) AS nb_agents_distincts
FROM transactions;
```

NB_AGENTS_DISTINCTS

5

On a 20 transactions mais seulement **5 agents distincts**. `DISTINCT` élimine les doublons avant de compter.

Question business réelle : « Combien de clients différents ont effectué une transaction en janvier ? »

SQL

```
SELECT COUNT(DISTINCT id_client) AS clients_janvier
FROM transactions
WHERE mois = '2024-01';
```

CLIENTS_JANVIER

5

3.2 SUM() — Additionner

Rôle : calcule la somme totale d'une colonne numérique.

SQL

```
SELECT SUM(montant) AS chiffre_affaires_total
FROM transactions;
```

CHIFFRE_AFFAIRES_TOTAL

25 000

Avec un filtre WHERE :

SQL

```
SELECT SUM(montant) AS ca_abidjan
FROM transactions
WHERE region = 'Abidjan';
```

CA_ABIDJAN

11 000



SUM() ignore automatiquement les valeurs NULL — si une ligne a **montant = NULL**, elle est exclue du calcul.

3.3 AVG() — Calculer la moyenne

Rôle : calcule la moyenne arithmétique d'une colonne numérique.

SQL

```
SELECT AVG(montant) AS montant_moyen
FROM transactions;
```

MONTANT_MOYEN

1250.0

Problème : le résultat s'affiche avec des décimales. En reporting, on arrondit avec `ROUND()` :

SQL

```
SELECT ROUND(AVG(montant), 0) AS montant_moyen
FROM transactions;
```

MONTANT_MOYEN

1250

`ROUND(valeur, nb_décimales)` — `0` arrondit à l'entier, `2` garde 2 décimales. À utiliser systématiquement avec `AVG()`.

Piège classique de AVG avec les NULL :

Imaginons une colonne `note_satisfaction` avec 3 valeurs : `5`, `NULL`, `3`.

SQL

```
SELECT AVG(note_satisfaction) FROM avis;
-- Résultat : 4 (= (5 + 3) / 2 — le NULL est ignoré)
```

⚠ `AVG()` ignore les NULL — il divise par le **nombre de valeurs non nulles**, pas par le nombre total de lignes. Si 100 clients n'ont pas répondu à l'enquête satisfaction, AVG ne prend pas ça en compte. C'est souvent une erreur d'interprétation.

3.4 MIN() et MAX() — Trouver les extrêmes

Rôle : retourne la valeur minimale ou maximale d'une colonne.

SQL

```
SELECT
  MIN(montant) AS transaction_min,
  MAX(montant) AS transaction_max
FROM transactions;
```

TRANSACTION_MIN	TRANSACTION_MAX
500	2 000

Sur du texte : `MIN()` et `MAX()` fonctionnent aussi sur des colonnes texte — ils retournent la valeur la plus petite ou grande dans l'ordre alphabétique.

SQL

```
SELECT MIN(agent) AS premier_alpha, MAX(agent) AS dernier_alpha
FROM transactions;
```

3.5 Tableau récapitulatif

FONCTION	CE QU'ELLE FAIT	IGNORE NULL ?	EXEMPLE BUSINESS
<code>COUNT(*)</code>	Nombre total de lignes	Non	Combien de transactions ?
<code>COUNT(col)</code>	Nombre de valeurs non nulles	Oui	Combien de clients ont un email ?
<code>COUNT(DISTINCT col)</code>	Nombre de valeurs uniques	Oui	Combien de clients distincts ?
<code>SUM(col)</code>	Somme totale	Oui	Chiffre d'affaires total
<code>AVG(col)</code>	Moyenne	Oui	Montant moyen par transaction
<code>ROUND(AVG(col), n)</code>	Moyenne arrondie	Oui	Panier moyen lisible
<code>MIN(col)</code>	Valeur la plus petite	Oui	Transaction la plus basse
<code>MAX(col)</code>	Valeur la plus grande	Oui	Transaction la plus haute

4. GROUP BY — Regrouper pour comparer

L'idée intuitive

Seule, une agrégation comme `SUM(montant)` te donne **un seul chiffre** pour toute la table.

Mais ton DG ne veut pas *le total global* — il veut *le total **par agent**, le total **par région***, etc.

`GROUP BY` découpe la table en **sous-groupes** avant d'appliquer la fonction d'agrégation à chaque groupe séparément.

💡 Si tu viens d'Excel : `GROUP BY` + `SUM()`, c'est l'équivalent de `SOMME.SI.ENS()` — ou encore mieux, d'un **Tableau Croisé Dynamique** entier, en une seule requête. Sauf qu'ici, ça marche sur 10 millions de lignes sans planter.

Visualisation — avant/après GROUP BY

Table originale (extrait) :

agent	montant
Koné Mamadou	2 000
Traoré Aminata	500
Koné Mamadou	500
Kouassi Éric	1 000
Traoré Aminata	2 000
Koné Mamadou	1 500

Après `GROUP BY agent` + `SUM(montant)` :

GROUP 1 : Koné Mamadou	→ 2000 + 500 + 1500	= 4 000
GROUP 2 : Traoré Aminata	→ 500 + 2000	= 2 500
GROUP 3 : Kouassi Éric	→ 1000	= 1 000

Résultat final :

AGENT	TOTAL_VENTES
Koné Mamadou	4 000
Traoré Aminata	2 500
Kouassi Éric	1 000

Syntaxe de base

SQL

```
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
GROUP BY agent;
```

La règle d'or de GROUP BY

Toute colonne dans le **SELECT** qui n'est PAS une fonction d'agrégation DOIT apparaître dans le **GROUP BY**.

✅ Correct :

SQL

```
SELECT agent, region, SUM(montant) AS total
FROM transactions
GROUP BY agent, region;
-- agent ET region sont dans le GROUP BY
```

❌ Erreur classique :

SQL

```
SELECT agent, region, SUM(montant) AS total
FROM transactions
GROUP BY agent;
-- region est dans le SELECT mais PAS dans le GROUP BY → erreur
```

💡 Pourquoi cette règle ? Si tu groupes par `agent`, SQL crée une ligne par agent. Mais si tu ajoutes `region` dans le `SELECT` sans la grouper, SQL ne sait pas quelle région afficher pour Koné Mamadou qui a travaillé à Abidjan. C'est ambigu → erreur.

Exemples progressifs

Exemple 1 — Chiffre d'affaires par région :

SQL

```
SELECT region, SUM(montant) AS chiffre_affaires
FROM transactions
GROUP BY region
ORDER BY chiffre_affaires DESC;
```

REGION	CHIFFRE_AFFAIRES
Abidjan	11 000
Bouaké	5 000
Yamoussoukro	4 500
San Pedro	4 500

Exemple 2 — Nombre de transactions par type de produit :

SQL

```
SELECT type_produit, COUNT(*) AS nb_transactions
FROM transactions
GROUP BY type_produit
ORDER BY nb_transactions DESC;
```

TYPE_PRODUIT	NB_TRANSACTIONS
Recharge_500	6
Forfait_internet	6
Recharge_1000	4
Forfait_voix	4

Exemple 3 — Statistiques complètes par agent :

SQL

```
SELECT
  agent,
  COUNT(*) AS nb_transactions,
  SUM(montant) AS total_ventes,
  ROUND(AVG(montant), 0) AS montant_moyen,
  MIN(montant) AS vente_min,
  MAX(montant) AS vente_max
FROM transactions
GROUP BY agent
ORDER BY total_ventes DESC;
```

AGENT	NB_TRANSACTIONS	TOTAL_VENTES	MONTANT_MOYEN	VENTE_MIN	VENTE_MAX
Koné Mamad ou	5	6 500	1 300	500	2 000
Traoré Aminata	4	5 000	1 250	500	2 000
N'Guess an Fatou	4	4 500	1 125	500	1 500
Kouassi Éric	4	4 500	1 125	500	2 000
Bamba Seydou	3	4 500	1 500	1 000	2 000

💡 Ce tableau, c'est exactement ce qu'un DA produit pour un comité de direction. 5 lignes, tout est là.

Exemple 4 — GROUP BY sur deux colonnes :

```
SQL
SELECT agent, mois, SUM(montant) AS ventes_du_mois
FROM transactions
GROUP BY agent, mois
ORDER BY agent, mois;
```

→ Tu obtiens une ligne par combinaison agent + mois — parfait pour suivre l'évolution mensuelle de chaque agent.

5. HAVING — Filtrer les groupes

Le problème

Tu veux savoir quels agents ont généré **plus de 4 000 FCFA** de ventes. Tu essaies :

```
SQL
-- ❌ ERREUR - ne fonctionne pas
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
WHERE SUM(montant) > 4000
GROUP BY agent;
```

Pourquoi ça ne marche pas ? Parce que `WHERE` filtre les **lignes brutes**, avant que les groupes soient formés. À ce stade, `SUM(montant)` n'existe pas encore.

C'est pour ça qu'il existe `HAVING`.

WHERE vs HAVING — le schéma mental

Étape 1 – WHERE filtre les LIGNES (données brutes)
↓
Étape 2 – GROUP BY regroupe
↓
Étape 3 – HAVING filtre les GROUPES (résultats agrégés)

	WHERE	HAVING
Agit sur	Lignes individuelles	Groupes résultants
Quand	Avant le GROUP BY	Après le GROUP BY
Peut filtrer sur	Colonnes brutes	Fonctions d'agrégation
Exemple	<code>WHERE region = 'Abidjan'</code>	<code>HAVING SUM(montant) > 4000</code>

Syntaxe

```
SQL
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
GROUP BY agent
HAVING SUM(montant) > 4000;
```

AGENT	TOTAL_VENTES
Koné Mamadou	6 500
Traoré Aminata	5 000

Exemples progressifs

Exemple 1 — Régions avec plus de 4 transactions :

SQL

```
SELECT region, COUNT(*) AS nb_transactions
FROM transactions
GROUP BY region
HAVING COUNT(*) > 4;
```

REGION	NB_TRANSACTIONS
Abidjan	9

Exemple 2 — WHERE et HAVING combinés :

Question : Quels agents d'Abidjan ont vendu plus de 3 000 FCFA ?

SQL

```
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
WHERE region = 'Abidjan'
GROUP BY agent
HAVING SUM(montant) > 3000
ORDER BY total_ventes DESC;
```

AGENT	TOTAL_VENTES
Koné Mamadou	6 500
Kouassi Éric	4 500

- ✓ `WHERE region = 'Abidjan'` → filtre d'abord les lignes (garde seulement Abidjan)
- ✓ `GROUP BY agent` → regroupe les lignes d'Abidjan par agent
- ✓ `HAVING SUM(montant) > 3000` → garde seulement les agents au-dessus du seuil

Exemple 3 — Produits vendus en moyenne à plus de 1 200 FCFA :

SQL

```
SELECT type_produit, ROUND(AVG(montant), 0) AS montant_moyen
FROM transactions
GROUP BY type_produit
HAVING AVG(montant) > 1200;
```

TYPE_PRODUIT	MONTANT_MOYEN
Forfait_internet	2 000
Forfait_voix	1 500

6. L'ordre d'exécution SQL — pourquoi certaines requêtes plantent

C'est le concept le plus incompris par les débutants — et celui qui explique la quasi-totalité des erreurs mystérieuses.

SQL n'exécute pas les clauses dans l'ordre où tu les écris. Il les traite dans cet ordre précis :

1. FROM → Quelle table ?
2. WHERE → Quelles lignes brutes ?
3. GROUP BY → Quels groupes ?
4. HAVING → Quels groupes à garder ?
5. SELECT → Quelles colonnes afficher ?
6. ORDER BY → Dans quel ordre ?
7. LIMIT → Combien de lignes ?

Ce que ça explique

Pourquoi `WHERE SUM(montant) > 4000` ne marche pas ?

→ `WHERE` est l'étape 2. `GROUP BY` est l'étape 3. Au moment où `WHERE` s'exécute, les groupes n'ont pas encore été formés — `SUM(montant)` n'existe pas encore. C'est pour ça qu'il faut `HAVING` (étape 4).

Pourquoi on ne peut pas utiliser un alias dans `HAVING` en SQLite ?

SQL

```
-- ✖ ERREUR en SQLite
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
GROUP BY agent
HAVING total_ventes > 4000;

-- total_ventes est créé à l'étape 5 (SELECT)
-- mais HAVING s'exécute à l'étape 4 - l'alias n'existe pas encore

-- ✔ Correct : utiliser la fonction complète
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
GROUP BY agent
HAVING SUM(montant) > 4000;
```

⚠ **Note** : certains moteurs modernes (PostgreSQL, BigQuery, MySQL récent) autorisent les alias dans **HAVING**. Mais en SQLite (notre environnement), ça ne marche pas. Utilise toujours la fonction complète dans **HAVING**.

Pourquoi **ORDER BY** peut utiliser les alias ?

SQL

```
-- ✔ Correct - ORDER BY vient après SELECT
SELECT agent, SUM(montant) AS total_ventes
FROM transactions
GROUP BY agent
ORDER BY total_ventes DESC;
```

→ **ORDER BY** (étape 6) s'exécute après **SELECT** (étape 5) — l'alias **total_ventes** existe déjà.

Mémo visuel

Ordre d'écriture	→	Ordre d'exécution réel
SELECT	←	5. SELECT
FROM	←	1. FROM ← début
WHERE	←	2. WHERE
GROUP BY	←	3. GROUP BY
HAVING	←	4. HAVING
ORDER BY	←	6. ORDER BY
LIMIT	←	7. LIMIT ← fin

💡 *Mémore ce ordre. La prochaine fois qu'une requête SQL plante avec un message d'erreur bizarre, demande-toi à quelle étape se produit le problème.*

7. Tout combiner — requêtes réalistes

On assemble maintenant toutes les pièces pour répondre à des vraies questions business.

Question 1 — Top 3 des agents par chiffre d'affaires

« Quels sont nos 3 meilleurs agents ce trimestre ? »

SQL

```
SELECT
  agent,
  COUNT(*) AS nb_ventes,
  SUM(montant) AS chiffre_affaires
FROM transactions
GROUP BY agent
ORDER BY chiffre_affaires DESC
LIMIT 3;
```

AGENT	NB_VENTES	CHIFFRE_AFFAIRES
Koné Mamadou	5	6 500
Traoré Aminata	4	5 000
Kouassi Éric	4	4 500

Question 2 — Régions sous-performantes

« Quelles régions ont généré moins de 5 000 FCFA de ventes ? »

SQL

```
SELECT
  region,
  SUM(montant) AS total_ventes,
  COUNT(*) AS nb_transactions
FROM transactions
GROUP BY region
HAVING SUM(montant) < 5000
ORDER BY total_ventes ASC;
```

REGION	TOTAL_VENTES	NB_TRANSACTIONS
San Pedro	4 500	4
Yamoussoukro	4 500	3

Question 3 — Évolution mensuelle du CA

« Comment le chiffre d'affaires a-t-il évolué de janvier à mars ? »

SQL

```
SELECT
  mois,
  SUM(montant) AS ca_mensuel,
  COUNT(*) AS nb_transactions
FROM transactions
GROUP BY mois
ORDER BY mois ASC;
```

MOIS	CA_MENSUEL	NB_TRANSACTIONS
2024-01	5 500	5
2024-02	8 000	6
2024-03	11 500	9

Insight : Le CA a plus que doublé entre janvier et mars — est-ce une tendance durable ou un pic ?
À approfondir.

Question 4 — Analyse croisée agent × mois (périmètre Abidjan)

« Pour les agents d'Abidjan, quelles sont leurs ventes mensuelles ? »

SQL

```
SELECT
  agent,
  mois,
  SUM(montant) AS ventes
FROM transactions
WHERE region = 'Abidjan'
GROUP BY agent, mois
ORDER BY agent, mois;
```

AGENT	MOIS	VENTES
Koné Mamadou	2024-01	2 500
Koné Mamadou	2024-02	1 500
Koné Mamadou	2024-03	2 500
Kouassi Éric	2024-01	1 000
Kouassi Éric	2024-02	1 000
Kouassi Éric	2024-03	2 500

8. Les pièges classiques

Piège 1 — COUNT(*) vs COUNT(colonne)

SQL

```
-- Table avec 5 lignes, dont 2 ont agent = NULL
```

```
SELECT COUNT(*)      FROM transactions; -- Résultat : 5
```

```
SELECT COUNT(agent)  FROM transactions; -- Résultat : 3
```

⚠ Si tu veux compter les lignes non nulles d'une colonne spécifique → **COUNT(colonne)**.

Si tu veux compter toutes les lignes d'une table → **COUNT(*)**.

Piège 2 — AVG ne prend pas en compte les absences

Imaginons 10 agents : 8 ont vendu, 2 n'ont rien vendu (montant = NULL car aucune transaction).

SQL

```
SELECT AVG(montant) FROM transactions;
-- Divise la somme par 8, pas par 10
-- La moyenne est surestimée
```

- ✅ Si tu veux inclure les absences dans ta moyenne, remplace les NULL par 0 avant de calculer — ou adapte ta question (« montant moyen par transaction » ≠ « montant moyen par agent »).

Piège 3 — Oublier une colonne dans GROUP BY

SQL

```
-- ❌ ERREUR en SQLite strict
SELECT agent, region, SUM(montant)
FROM transactions
GROUP BY agent;
-- region est dans SELECT mais pas dans GROUP BY

-- ✅ Correct
SELECT agent, region, SUM(montant)
FROM transactions
GROUP BY agent, region;
```

Piège 4 — Utiliser WHERE au lieu de HAVING

SQL

```
-- ❌ ERREUR
SELECT agent, SUM(montant) AS total
FROM transactions
WHERE SUM(montant) > 4000
GROUP BY agent;

-- ✅ Correct
SELECT agent, SUM(montant) AS total
FROM transactions
GROUP BY agent
HAVING SUM(montant) > 4000;
```

Piège 5 — Confondre AVG et SUM dans la lecture

SQL

```
SELECT region, ROUND(AVG(montant), 0) AS montant_moyen
FROM transactions
GROUP BY region;
```

REGION	MONTANT_MOYEN
Abidjan	1 222

⚠ Ce chiffre, c'est **la valeur moyenne d'une transaction** à Abidjan — pas le CA total d'Abidjan (qui est 11 000). Confondre les deux est une erreur courante dans les présentations aux décideurs. Toujours bien nommer ses colonnes et vérifier ce qu'on mesure.

9. Cheatsheet — Agrégations et GROUP BY

Les fonctions d'agrégation

SQL

```
COUNT(*)           -- Nombre total de lignes
COUNT(colonne)    -- Lignes avec valeur non nulle
COUNT(DISTINCT col) -- Valeurs uniques
SUM(colonne)        -- Somme
AVG(colonne)        -- Moyenne (ignore les NULL)
ROUND(AVG(colonne), 0) -- Moyenne arrondie
MIN(colonne)        -- Valeur minimale
MAX(colonne)        -- Valeur maximale
```

Structure complète avec toutes les clauses

```
SQL
SELECT  colonne, AGG_FONCTION(colonne) AS alias
FROM    table
WHERE   condition_sur_lignes
GROUP BY colonne
HAVING  condition_sur_groupes
ORDER BY alias DESC
LIMIT   N;
```

WHERE vs HAVING en un coup d'œil

```
SQL
WHERE  region = 'Abidjan'      -- Filtre les lignes brutes
HAVING SUM(montant) > 4000     -- Filtre les groupes calculés
```

Ordre d'exécution à retenir

FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT

Requêtes types

```
SQL
-- Comptage par catégorie
SELECT categorie, COUNT(*) AS n FROM t GROUP BY categorie;

-- Top N par total
SELECT col, SUM(val) AS total FROM t GROUP BY col ORDER BY total DESC LIMIT N;

-- Groupes au-dessus d'un seuil
SELECT col, SUM(val) AS total FROM t GROUP BY col HAVING SUM(val) > seuil;

-- Filtre + regroupement
SELECT col, ROUND(AVG(val), 0) AS moy FROM t WHERE condition GROUP BY col;
```

10. Quiz — Vérifie ta compréhension

Réponds mentalement, puis clique sur "Voir la réponse" pour te corriger.

Q1. Quelle requête compte le nombre de régions **distinctes** dans la table `transactions` ?

- a) `SELECT COUNT(*) FROM transactions;`
- b) `SELECT COUNT(DISTINCT region) FROM transactions;`
- c) `SELECT COUNT(region) FROM transactions;`

 Voir la réponse

 **b)** — `COUNT(*)` compte toutes les lignes (20). `COUNT(region)` compte les lignes où region n'est pas NULL. Seul `COUNT(DISTINCT region)` compte les **4 régions uniques** : Abidjan, Bouaké, San Pedro, Yamoussoukro.

Q2. Tu veux le chiffre d'affaires total **par région**. Quelle est la bonne requête ?

- a) `SELECT region, SUM(montant) FROM transactions;`
- b) `SELECT region, SUM(montant) FROM transactions GROUP BY region;`
- c) `SELECT SUM(montant) FROM transactions WHERE region GROUP BY region;`

 Voir la réponse

 **b)** — Sans `GROUP BY`, la réponse a) retourne une seule ligne avec le total global (comportement ambigu ou erreur selon le moteur SQL). La réponse c) a une syntaxe incorrecte dans le `WHERE`.

Q3. Quelle est la différence entre `WHERE` et `HAVING` ?

- a) `WHERE` filtre avant le regroupement, `HAVING` filtre après
- b) `WHERE` fonctionne uniquement avec les nombres, `HAVING` avec le texte
- c) Il n'y a aucune différence, les deux sont interchangeables

👉 Voir la réponse

✅ a) — `WHERE` filtre les lignes **avant** que les groupes soient formés. `HAVING` filtre les **groupes** après agrégation. C'est pourquoi on ne peut pas utiliser `SUM()` dans un `WHERE`.

Q4. Cette requête contient une erreur. Laquelle ?

SQL

```
SELECT agent, region, COUNT(*) AS nb
FROM transactions
GROUP BY agent;
```

- a) `COUNT(*)` ne peut pas être utilisé avec `GROUP BY`
- b) `region` est dans le `SELECT` mais absent du `GROUP BY`
- c) Il manque un `ORDER BY`

👉 Voir la réponse

✅ b) — Règle d'or : toute colonne non agrégée dans le `SELECT` doit apparaître dans le `GROUP BY`.
Correction : `GROUP BY agent, region;`

Q5. Dans quel ordre SQL exécute-t-il ces clauses ?

- a) SELECT → FROM → WHERE → GROUP BY → HAVING → ORDER BY
- b) FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY
- c) FROM → GROUP BY → WHERE → HAVING → SELECT → ORDER BY

👉 Voir la réponse

✅ b) — L'ordre réel d'exécution est : FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY → LIMIT. C'est l'inverse de l'ordre d'écriture pour les premières clauses. Cet ordre explique pourquoi on ne peut pas utiliser un alias du SELECT dans un HAVING en SQLite.

Q6. Tu veux les agents qui ont effectué **strictement plus de 3 transactions**. Quelle est la bonne requête ?

SQL

```
-- Option A
SELECT agent, COUNT(*) AS nb
FROM transactions
WHERE COUNT(*) > 3
GROUP BY agent;

-- Option B
SELECT agent, COUNT(*) AS nb
FROM transactions
GROUP BY agent
HAVING COUNT(*) > 3;
```

👉 Voir la réponse

✅ **Option B** — `COUNT(*)` est une fonction d'agrégation : elle n'existe qu'après le `GROUP BY`. La mettre dans un `WHERE` déclenche une erreur car `WHERE` s'exécute avant `GROUP BY`. Il faut `HAVING`.

Q7. La table `avis` a 10 lignes. La colonne `note` contient : 5, 4, NULL, 3, NULL, 5, 4, NULL, 3, 2. Que retourne `SELECT AVG(note) FROM avis;` ?

- a) 2.6 (somme 26 ÷ 10)
- b) 3.71 (somme 26 ÷ 7 valeurs non nulles)
- c) NULL

👉 Voir la réponse

✅ **b) 3.71** — `AVG()` ignore les NULL. Il calcule $(5+4+3+5+4+3+2) \div 7 = 26 \div 7 \approx 3.71$. Les 3 lignes NULL sont exclues. Si tu voulais inclure les absences comme des 0, tu devrais d'abord remplacer les NULL par 0.

Q8. Que fait exactement cette requête ?

SQL

```
SELECT type_produit, COUNT(*) AS nb_ventes, SUM(montant) AS total
FROM transactions
WHERE mois = '2024-03'
GROUP BY type_produit
HAVING COUNT(*) >= 2
ORDER BY total DESC;
```

- a) Elle retourne tous les produits vendus en mars 2024 avec leurs totaux
- b) Elle retourne les produits vendus **au moins 2 fois** en mars 2024, triés par CA décroissant
- c) Elle retourne les produits dont le total dépasse 2 000 FCFA en mars 2024

👉 Voir la réponse

✅ b) — Décortiquons : `WHERE mois = '2024-03'` filtre les transactions de mars. `GROUP BY type_produit` regroupe par produit. `HAVING COUNT(*) >= 2` garde seulement les produits vendus au moins 2 fois. `ORDER BY total DESC` trie par CA décroissant. La réponse a) est fausse (le HAVING filtre les produits rares). La réponse c) est fausse (HAVING porte sur COUNT, pas SUM).

11. Résumé du module

CONCEPT	CE QU'IL FAIT	EXEMPLE
<code>COUNT(*)</code>	Compte toutes les lignes	Nombre de transactions
<code>COUNT(col)</code>	Compte les valeurs non nulles	Clients avec email renseigné
<code>COUNT(DISTINCT col)</code>	Compte les valeurs uniques	Nombre de régions distinctes
<code>SUM(col)</code>	Somme totale	Chiffre d'affaires
<code>ROUND(AVG(col), 0)</code>	Moyenne arrondie	Panier moyen lisible
<code>MIN(col)</code> / <code>MAX(col)</code>	Extrêmes	Transaction min et max
<code>GROUP BY col</code>	Regroupe par catégorie	CA par région
<code>HAVING condition</code>	Filtre les groupes	Agents avec CA > 4 000
<code>WHERE</code> vs <code>HAVING</code>	WHERE = avant GROUP BY / HAVING = après	Voir section 5
Ordre d'exécution	FROM → WHERE → GROUP BY → HAVING → SELECT → ORDER BY	Évite les erreurs mystérieuses

→ Module suivant

Tu sais maintenant résumer et comparer des données en SQL. Mais toutes les informations ne sont pas dans **une seule table**.

Dans le **Module 08**, on apprend à relier plusieurs tables entre elles avec les **jointures** — `INNER JOIN`, `LEFT JOIN`, `RIGHT JOIN`. C'est là que SQL révèle toute sa puissance.

→ Module 08 — Jointures : relier les tables